

Core Language Working Group "ready" Issues for the November, 2016 (Issaquah) meeting

Section references in this document reflect the section numbering of document [WG21 N4582](#).

1395. Partial ordering of variadic templates reconsidered

Section: 14.8.2.5 [temp.deduct.type] **Status:** ready **Submitter:** John Spicer **Date:** 2011-09-21

The resolution of issue 692 (found in document N3281) made the following example ambiguous and thus ill-formed:

```
template<class T>
void print(ostream &os, const T &t) {
    os << t;
}

template <class T, class... Args>
void print(ostream &os, const T &t, const Args&... rest) {
    os << t << ", ";
    print(os, rest...);
}

int main() {
    print(cout, 42);
    print(cout, 42, 1.23);
}
```

This pattern seems fairly intuitive; is it reason to reconsider or modify the outcome of issue 692?

(See also issue 1432.)

Notes from the October, 2012 meeting:

CWG agreed that the example should be accepted, handling this case as a late tiebreaker, preferring an omitted parameter over a parameter pack.

Additional note (March, 2013):

For another example:

```
template<typename ...T> int f(T*...) {
    return 1;
}
template<typename T> int f(const T&) {
    return 2;
}
int main() {
    if (f((int*)0) != 1) {
        return 1;
    }
    return 0;
}
```

This worked as expected prior to the resolution of issue 692.

Proposed resolution (June, 2016):

1. Change 14.8.2.4 [temp.deduct.partial] paragraph 8 as follows:

~~If A was transformed from a function parameter pack and P is not a parameter pack, type deduction fails. Otherwise, using~~ **Using** the resulting types P and A, the deduction is then done as described in 14.8.2.5 [temp.deduct.type]. If P is a function parameter pack, the type A of each remaining parameter type of the argument template is compared with the type P of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. **Similarly, if A was transformed from a function parameter pack, it is compared with each remaining parameter type of the parameter template.** If deduction succeeds for a given type, the type from the argument template is considered to be at least as specialized as the type from the parameter template. [Example:...

2. Add the following as a new paragraph following 14.8.2.4 [temp.deduct.partial] paragraph 10:

Function template F is at least as specialized as function template G if, for each pair of types used to determine the ordering, the type from F is at least as specialized as the type from G. F is more specialized than G if F is at least as specialized as G and G is not at least as specialized as F.

If, after considering the above, function template F is at least as specialized as function template G and vice-versa, and if G has a trailing parameter pack for which F does not have a corresponding parameter, and if F does not have a trailing parameter pack, then F is more specialized than G.

This resolution also resolves issue 1825.

1825. Partial ordering between variadic and non-variadic function templates

Section: 14.8.2.4 [temp.deduct.partial] **Status:** ready **Submitter:** Steve Clamage **Date:** 2013-12-30

Given the following example,

```
template <class ...T> int f(T*...) { return 1; }
template <class T> int f(const T&) { return 2; }
void g() {
    f((int*)0);
}
```

the current specification makes the call ambiguous because deduction fails in both directions: with A being τ and P being τ^* in one direction and A being τ^* and P being τ , because 14.8.2.4 [temp.deduct.partial] paragraph 8 says,

If A was transformed from a function parameter pack and P is not a parameter pack, type deduction fails.

It is not clear whether this is the best outcome, however; it might be better to consider the first template more specialized, with the variadic/non-variadic test being a tie-breaker if there is no other reason to prefer one over the other based on the parameter types.

Notes from the February, 2014 meeting:

CWG felt that the best approach would be, when comparing P and A, if A is a pack and P is not, A should be repeated for each remaining instance of P and then use the variadic/nonvariadic criterion as a late tiebreaker if the result is still ambiguous. This would apply in the general case (including 14.8.2.4 [temp.deduct.partial]), not just in function calls.

Proposed resolution (June, 2016):

This issue is resolved by the resolution of issue 1395.

1961. Potentially-concurrent actions within a signal handler

Section: 1.10 [intro.multithread] **Status:** ready **Submitter:** Faisal Vali **Date:** 2014-07-04

According to 1.10 [intro.multithread] paragraph 23,

Two actions are *potentially concurrent* if

- they are performed by different threads, or
- they are unsequenced, and at least one is performed by a signal handler.

This definition should exclude the case when both actions are performed by a signal handler.

Notes from the October, 2015 meeting:

SG1 agrees that the existing wording should be amended to say something like “and they are not both performed by the same signal handler invocation.”

Proposed resolution (November, 2015):

Change 1.10 [intro.multithread] paragraph 23 as follows:

Two actions are *potentially concurrent* if

- they are performed by different threads, or
- they are unsequenced, and at least one is performed by a signal handler, **and they are not both performed by the same signal handler invocation.**

2143. Value-dependency via injected-class-name

Section: 14.6.2.1 [temp.dep.type] **Status:** ready **Submitter:** Maxim Kartashev **Date:** 2015-06-16

There does not appear to be a rule that causes `p` or `this` to be dependent in the following example:

```
template <typename T> struct A {
    void foo() {
        A* p = 0;
        bar(p);    // will be found by ADL at the point of instantiation
        bar(this); // same here
    }
};

void bar(...);

int main() {
    A<int> a;
    a.foo();
}
```

Proposed resolution (February, 2016) [SUPERSEDED]:

Change 14.6.2.1 [temp.dep.type] bullet 9.7 as follows:

A type is dependent if it is

- ...
- **an injected-type-name (Clause 9 [class]) of a class template or** a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion, or
- denoted by...

Proposed resolution (March, 2016):

Change 14.6.2.1 [temp.dep.type] bullet 9.7 as follows:

A type is dependent if it is

- ...
- a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion **[Note: This includes an injected-class-name (Clause 9 [class]) of a class template used without a**

template-argument-list. —end note], or

- denoted by...

2155. Defining classes and enumerations via *using-declarations*

Section: 7.3.1.2 [namespace.memdef] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-07-05

The resolution of issue 1838 was intended to resolve issue 1021 but does not actually do so, as it applies only to declarations with a *declarator-id*, which is not true of classes and enumerations.

Proposed resolution (October, 2015):

1. Change 7.3.1.2 [namespace.memdef] paragraph 1 as follows:

A declaration in a namespace *N* (excluding declarations in nested scopes) whose *declarator-id* is an *unqualified-id* (8.3 [dcl.meaning]), whose *class-head-name* (Clause 9 [class]) or *enum-head-name* (7.2 [dcl.enum]) is an *identifier*, or whose *elaborated-type-specifier* is of the form *class-key attribute-specifier-seq_{opt} identifier* (7.1.6.3 [dcl.type.elab]), or that is an *opaque-enum-declaration*, declares (or redeclares) *its unqualified-id or identifier* as a member of *N*, and may be a definition. [Note:...

2. Change the grammar of 7.2 [dcl.enum] as follows:

enum-head:

enum-key attribute-specifier-seq_{opt} identifier enum-head-name_{opt} enum-base_{opt}
enum-key attribute-specifier-seq_{opt} nested-name-specifier identifier enum-base_{opt}

enum-head-name:

nested-name-specifier_{opt} identifier

2271. Aliasing *this*

Section: 12.1 [class.ctor] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-06-20

The restrictions against aliasing *this* inside a constructor should apply to all objects, not just to *const* objects.

Proposed resolution (June, 2016):

Change 12.1 [class.ctor] paragraph 12 as follows:

During the construction of a *const* *an* object, if the value of the object or any of its subobjects is accessed through a glvalue that is not obtained, directly or indirectly, from the constructor's *this* pointer, the value of the object or subobject thus obtained is unspecified. [Example:

```
struct C;
void no_opt(C*);

struct C {
    int c;
    C() : c(0) { no_opt(this); }
};

const C cobj;

void no_opt(C* cptr) {
    int i = cobj.c * 100; // value of cobj.c is unspecified
    cptr->c = 1;
    cout << cobj.c * 100 // value of cobj.c is unspecified
         << '\n';
}
```

```
extern struct D d;
struct D {
    D(int a) : a(a), b(d.a) {}
}
```

```
int a, b;
```

```
};
```

```
D d = D(1); // value of d.b is unspecified
```

—end example]